Lifecycle hooks, in nine moments

Animated explainer · src/hooks.js

A fixed loop is readable, but not extensible. This is the smallest extension point that fixes it: a function attached to a named moment, called by the loop, able to observe, rewrite, or block. Ten scenes, about two minutes, no key and no network.

10 scenes · 2:22 watch time · 9 lifecycle moments · 4 can change the run · 0 edits to the loop

CONTENTS

What is in this explainer

01 · Why hooks exist 3

02 · The contract 4

03 · The lifecycle 5

04 · The first two moments 6

05 · Inside one step 7

06 · The deciding hooks 8

07 · How a run ends 9

08 · The registry 10

09 · Putting it to work 11

10 · Recap 12

Page numbers refer to the folio at the foot of each page.

01 · Why hooks exist

A fixed loop is readable. It is not extensible.

SRC/AGENT.JS · *the loop*

```
for (let step = 1; step <= maxSteps; step++) {
  const reply = await chat(messages, tools);
  if (reply.toolCalls.length === 0) return reply;
  // ...dispatch the tools, feed results back...
}
```

- 📝 Log every command edit the loop
- 🔐 Never touch .env edit the loop
- 🔔 Tell me when it gives up edit the loop

> **the fix** A hook is a function attached to a named moment in the loop.
> **On screen:** Every coding agent is a loop: ask the model, run the tools it asks for, feed the results back, repeat. That loop is fixed on purpose. But every new rule — log every command, protect .env — used to mean editing it. A hook fixes that. It is a function attached to a named moment, called by the loop, never by the model.

02 · The contract

The whole API is one return value.

payload in { name, args, step } a plain object — no model, no tools

fn your hook called by the loop

decision out undefined nothing changes

undefined observe only no effect on the run

{ note } say something printed for whoever listens

{ task } rewrite the task UserPromptSubmit only

{ replace } replace the result PostToolUse only

{ block, reason } refuse the call SessionStart · UserPromptSubmit · PreToolUse

> A hook does not call the model and does not run tools. It is a pure function of the payload.
> **On screen:** Here is the whole contract. The loop hands your hook a payload object. Return nothing and the run is untouched. Return a note and you have said something, still without changing anything. Two return shapes can change what happens: rewrite the task, or replace the observation the model reads. The last shape is a refusal. Return block with a reason, and the tool never runs.

03 · The lifecycle

One run, nine moments.

once, before the first request

1 🚀 SessionStart

2 ✍️ UserPromptSubmit

every step↻

3 🧠 PreModel

model call

4 💬 PostModel

↳

5 🛠️ PreToolUse

tool

6 📦 PostToolUse

the loop goes round with the observation in hand

ending

7 🏁 Stop

8 ⏳ StepLimit

9 🔒 SessionEnd

always

4 can change the run 5 only observe the diagram is generated from `HOOK_EVENTS`

> **On screen:** One run fires nine moments. First, SessionStart: the run exists, but nothing has been sent yet. Then UserPromptSubmit, and the loop begins: pre-model, post-model, pre-tool, post-tool — every step. The run ends one of two ways: Stop with an answer, or StepLimit when the budget runs out. Whichever way it went, SessionEnd always fires. Nine moments; four of them can change the run.

04 · The first two moments

Before the model sees anything.

🚀 1 SessionStart once receives task · workspace · config · log may refuse to start · post a note load project rules · open a transcript · check the budget · print a greeting

✍️ 2 UserPromptSubmit once receives task may rewrite the task · block the run before fix the bug ✍️ hook returns { task } after fix the bug + always run the tests

> The model never sees the user's raw words. It sees the task after UserPromptSubmit has had its turn.
> **On screen:** SessionStart is where project rules are loaded, a transcript is opened, or a budget is checked. UserPromptSubmit is the user's words before the model has seen them. Rewrite the task here and the model only ever sees the new one. So the model never sees the raw prompt. It sees the prompt after the hook has had its turn.

05 · Inside one step

The heartbeat of the run.

3. **PreModel** — 6 messages

model call

4. **PostModel** — tool_calls

5. **PreToolUse** — run_command

tool runs

6. **PostToolUse** — exit code 0

NODE SRC/CLI.JS --DEMO · *text*

```
🧠 [HOOK:PreModel]     before every model call — step 3/12 · 6 messages
💬 [HOOK:PostModel]    after every reply — step 3 · tool_calls
🛠️ [HOOK:PreToolUse]   before every tool call — run_command {...}
📦 [HOOK:PostToolUse]  after every tool call — run_command · exit code 0
```

These four events repeat on every step. The hook trace is on by default, so you can watch them fire.

> **On screen:** Inside a step, before the request goes over the wire, PreModel lets you measure, summarise, or log the messages. After the reply, PostModel sees the answer or the tool call first. Then the tool call reaches PreToolUse. The tool runs, PostToolUse sees the result, and the loop goes round with the observation in hand. These four moments repeat on every step. They are the heartbeat of the run.

06 · The deciding hooks

Two verbs that change the run.

BLOCK PreToolUse tool call git push origin main hook { block: true, reason: 'no pushes' } model sees DENIED: ... Try another approach. The tool never runs. The model gets the refusal as an observation and can choose another way.

REPLACE PostToolUse tool output OPENAI_API_KEY=sk-or-abc123 hook { replace: '... sk-or-[redacted]' } model sees OPENAI_API_KEY=sk-or-[redacted] The result exists and the model has not seen it yet. Replace it: redact, cap, or annotate a failure.

> A refusal is an observation , not an exception. The loop cannot crash on a “no”, and the model gets another turn.
> **On screen:** PreToolUse is the one hook that changes what the agent is allowed to do. A permission layer lives here. PostToolUse changes what the model reads back. Redact a secret, cap a huge output, annotate a failure. The difference matters: a refusal comes back as a tool result, not an exception. The model gets another turn.

07 · How a run ends

Every path ends in the finally block.

🏁 Stop the model answered and the loop is ending answer · steps · messages A summary, a cost report, or the notification that says the job is done.

⏳ StepLimit the loop hit maxSteps without an answer steps · messages Failures need a lifecycle too: escalate, or dump the transcript for a post-mortem.

🔒 SessionEnd always — finished or not ok · reason · steps · durationMs · messages One place that runs whatever happened is what makes cleanup, timing and audit logs reliable rather than hopeful.

> **On screen:** Stop is the happy ending: no tool calls, an answer, a return. Good for a summary, a cost report, or a notification. StepLimit is the failure lifecycle. The run is already over, but you can escalate or dump the transcript. SessionEnd is the finally block. It runs whatever happened — which is what makes cleanup and audit logs reliable.

08 · The registry

A map, a list, and one answer.

SRC/HOOKS.JS · *run*

```
for (const handler of [...handlers]) {
  try {
    decision = await handler.fn(payload, { event });
  } catch (err) {
    out.errors.push(`${handler.id}: ${err.message}`);
    continue; // one bad hook is not the loop's problem
  }
  if (!decision || typeof decision !== 'object') continue;
  if (decision.block) { out.blocked = true; break; }
}
```

handlers run in registration order

A { note: 'checked' } → collected

B throws → reported, ignored

C { block: true } → first refusal wins

D skipped — no second opinion

WHAT RUN() RETURNS · *text*

```
{ event: 'PreToolUse', ran: 3, blocked: true, reason: 'no pushes',
  notes: ['checked'], errors: ['B: boom'] }
```

> Above all, every tool_call still gets a matching tool result — or the next request would be invalid.
> **On screen:** The registry is an ordinary map. on attaches a handler to an event name; a typo throws instead of silently never firing. run calls the handlers in order. A note is collected; the first block wins and the rest are skipped. If a handler throws, it is reported and ignored. One bad hook must not take the loop with it. Above all, every tool call still gets a matching result — or the next request would be invalid.

09 · Putting it to work

A permission layer in twelve lines.

1 Permission layer hooks.on('PreToolUse', ({ name, args }) => name === 'run_command' && /\bgit push\b/.test(args) ? { block: true, reason: 'no pushes' } : undefined); The model asks. The hook decides.

2 Redaction hooks.on('PostToolUse', ({ result }) => ({ replace: String(result) .replace(/sk-or-\S+/g, 'sk-or-[redacted]') })); Whatever was printed, a key never reaches the model.

3 Cleanup hooks.on('SessionEnd', ({ ok, reason, durationMs }) => ({ note: `run ${ok ? 'ok' : reason} in ${durationMs}ms` })); Failures get a lifecycle too.

WHAT THE MODEL SEES · *text*

```
tool → run_command {"command":"git push origin main"}
result ← DENIED: a PreToolUse hook blocked this call (no pushes). Try another approach.
```

> **On screen:** A permission layer is three lines: inspect the arguments, refuse the call, tell the model why. A redactor is one return value: whatever the command printed, a key never reaches the model. Failures get a lifecycle too. SessionEnd can report what happened and how long it took. This is the whole promise: your code, attached to the loop, with no edits to the loop itself.

10 · Recap

Your code, the loop's moments.

• 9 lifecycle moments

• 4 can change the run

• 0 edits to agent.js

→

Return a decision, or nothing. undefined, note, task, replace, block.

→

A refusal is an observation. The tool never runs, but the model is told why.

→

A hook that throws is ignored. One bad hook never takes the loop with it.

→

First block wins. The rest of the handlers are skipped.

TRY IT — NO KEY, NO NETWORK · *text*

```
node src/cli.js --hooks      # the catalogue
node src/cli.js --demo       # the lifecycle, live
node src/cli.js --demo --no-hooks
```

A tool is how the model acts on the world. A hook is how you act on the loop.

> **On screen:** Nine moments. Four decisions. Zero edits to the loop. Return a decision or nothing. Notes print; blocks win; errors are ignored; every tool call still gets a result. Run the catalogue, then watch the trace fire on a real demo. A tool is how the model acts on the world. A hook is how you act on the loop.